01

Route

0. React Router Dom

■ 라이브러리 설치

- npm install react-router-dom

■ <Router>

- <BrowserRouter>: 웹 애플리케이션에서 가장 흔히 사용하는 라우터로, HTML5의 히스토리 API를 사용하여 URL을 관리

- <HashRouter>: 정적 파일만을 제공하는 서버나 백엔드 서버가 없는 정적 웹사이트 호스팅 환경에서 사용

01. <BrowserRouter>

- 애플리케이션을 감싸는 라우터의 기본 컴포넌트
- HTML5의 히스토리 API를 사용하여 URL을 관리

```
import { BrowserRouter } from 'react-router-dom';

function App() {
    return (
        <BrowserRouter>
            <MainRoutes />
        </BrowserRouter>
    );
}
```

02. <Routes>와 <Route>

- <Routes>는 여러 개의 <Route>를 감싸서 각 URL 경로에 맞는 컴포넌트를 렌더링
- <Routes>는 하위 <Route> 중 일치하는 path를 가진 <Route> 를 실행하고 나머지 <Route>는 무시

```
import { Routes, Route } from 'react-router-dom';
import Home from './Home';
import About from './About';

function MainRoutes() {
    return (
        <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/about" element={<About />} />
        </Routes>
    );
}
```

03. Route의 index 속성

■ 중첩된 라우트에서 기본 경로로 표시할 페이지를 설정

```
import { Routes, Route } from 'react-router-dom';

function App() {
    return (
        <Routes>
            <Route path="/" element={<Layout />}>
                <Route index element={<Home />} />
                 {/* 기본 페이지 */}
                <Route path="about" element={<About />} />
            </Route>
        </Routes>
    );
}
```

04. <Link>와 <NavLink>

- 두 태그 모두 페이지 간 이동을 위한 링크
  - <Link>: 기본 링크를 제공하며, 스타일이 없다.
  - <NavLink>: 활성 상태일 때 스타일을 다르게 지정할 수 있다.
  - <a>의 기본 기능인 페이지 리로딩을 하지 않는다.

```
import { Link, NavLink } from 'react-router-dom';

function Navbar() {
 return (
   <nav>
    <Link to="/">Home</Link>
    <NavLink to="/about“ ActiveClassName="active">
              About</NavLink>
   </nav>
    );
}
```

05. useNavigate

- 프로그래밍 방식으로 페이지 이동을 제어하는 훅.
- 이벤트 발생시 특정 페이지로 이동할 때 유용.
- 비동기 적으로 처리되면 현재 컴포넌트의 랜더링이 끝난 다음 경로를 변경.

```
import { useNavigate } from 'react-router-dom';

function Home() {
    const navigate = useNavigate();

    return (
        <button onClick={() => navigate('/about')}>
            Go to About
        </button>
    );
}
```

06. useParams

- URL 파라미터를 가져오는 훅.
- 특정 ID로 데이터를 불러올 때 유용.

```
import { useParams } from 'react-router-dom';

function Post() {
    const { postId } = useParams();

    return <h2>Post ID: {postId}</h2>;
}
```

- /post/:postId 경로에서 postId 파라미터를 가져올 수 있다.

07. Outlet 컴포넌트

- 중첩된 라우트를 렌더링할 때 자식 경로가 표시될 위치를 지정

```
import { Routes, Route, Outlet } from 'react-router-dom';
function Layout() {
    return (
        <div>
            <h1>Main Layout</h1>
            <Outlet />  {/* 자식 컴포넌트가 여기에 렌더링 */}
        </div>
    );}
function App() {
  return (
     <Routes>
       <Route path="/" element={<Layout />}>
           <Route path="about" element={<About />} />
           <Route path="contact" element={<Contact />} />
            </Route>
       </Routes>
    );}
```

08. useOutletContext()

- React Router에서 제공하는 특수 훅
- 중첩된 라우트 구조에서 부모 컴포넌트가 <Outlet />을 통해 전달한 데이터를 하위 라우트 컴포넌트들이 사용할 수 있도록 전달
- React의 Context API와 유사하지만, 특정 라우트 구조 내에서만 데이터를 공유하는 데 사용

08. useOutletContext()

```
function App() {
return (
 <BrowserRouter>
  <Routes>
    <Route path="parent" element={<ParentComponent />}>
     <Route path="child" element={<ChildComponent />}>
       <Route path="grandchild“
                   element={<GrandchildComponent/>} />
     </Route>
    </Route>
  </Routes>
 </BrowserRouter>
    );
}

export default App;
```

08. useOutletContext()

```
import React from 'react';
import { Outlet } from 'react-router-dom';

function ParentComponent() {
    const sharedData = "This data is shared with all
                        nested routes";

    return (
        <div>
            <h1>Parent Component</h1>
            <Outlet context={{ sharedData }} />
             {/* 데이터를 context로 전달 */}
        </div>
    );
}

export default ParentComponent;
```

08. useOutletContext()

```
import React from 'react';
import { Outlet, useOutletContext } from 'react-router-
dom';

function ChildComponent() {
    const { sharedData } = useOutletContext();
    // 상위 컴포넌트의 데이터를 받아옴

    return (
        <div>
            <h2>Child Component</h2>
            <p>Data from Parent: {sharedData}</p>
            <Outlet />
        {/* GrandchildComponent를 렌더링할 Outlet */}
        </div>
    );
}

export default ChildComponent;
```

08. useOutletContext()

```
import React from 'react';
import { useOutletContext } from 'react-router-dom';

function GrandchildComponent() {
    const { sharedData } = useOutletContext();
    // 상위 컴포넌트의 데이터를 받아옴

    return (
        <div>
            <h3>Grandchild Component</h3>
            <p>Data from Parent: {sharedData}</p>
        </div>
    );
}

export default GrandchildComponent;
```

02

Context API

01. Context API

- React에서 컴포넌트 트리 전체에 걸쳐 데이터를 쉽게 전달하기 위해 제공되는 기능.
- 일반적으로, 부모에서 자식으로만 데이터를 전달할 수 있지만, Context API를 사용하면 중간 컴포넌트들을 거치지 않고 전역적으로 데이터를 전달 가능.
- 전역 상태 관리가 필요할 때 사용하는 기능
  - 사용자 정보, 테마 설정, 인증 상태 등
- 여러 컴포넌트에서 공통으로 사용하는 데이터를 저장하고 사용 가능.

02. Context API의 주요 구성 요소

React.createContext()

- Context 객체를 생성.
- 생성된 Context는 데이터가 저장되는 공간을 제공.

Provider

- Context 데이터를 제공하는 컴포넌트
- Context의 값을 설정.(value 속성)
- 하위에 있는 모든 컴포넌트는 Provider를 통해 Context 데이터에 접근.

useContext

- 하위 컴포넌트에서 Context 데이터에 접근하기 위해 사용하는 React 훅.

03. Context API 사용예제

```
//UserContext.js

import React, { createContext, useState } from 'react';

export const UserContext = createContext();
// UserContext를 생성

// UserProvider 컴포넌트를 정의해 Provider로 사용
export const UserProvider = ({ children }) => {
    const [user, setUser] = useState({ name: "Alice",
                                       age: 25 });

    return (
        <UserContext.Provider value={{ user, setUser }}>
            {children}
        </UserContext.Provider>
    );
};
```

03. Context API 사용예제

```
//App.js

import React from 'react';
import { UserProvider } from './UserContext';
import Navbar from './Navbar';
import UserProfile from './UserProfile';

function App() {
    return (
        <UserProvider>
            <Navbar />
            <UserProfile />
        </UserProvider>
    );
}

export default App;
```

03. Context API 사용예제

```
// Navbar.js

import { useContext } from 'react';
import { UserContext } from './UserContext';

function Navbar() {
    const { user } = useContext(UserContext);
    // UserContext에서 user 정보 가져오기

    return (
        <nav>
            <h2>Welcome, {user.name}!</h2>
        </nav>
    );
}

export default Navbar;
```

03. Context API 사용예제

```
// UserProfile.js
import React, { useContext } from 'react';
import { UserContext } from './UserContext';
import UserInfo from './UserInfo';

export default function UserProfile() {
    const { user, setUser } = useContext(UserContext);
    // UserContext에서 user와 setUser 가져오기
    const updateUser = () => {
        setUser({ name: "Bob", age: 30 });
    // 사용자 정보 업데이트
    };
    return (
    <div>
      <h2>User Profile</h2>
      <UserInfo />
      <button onClick={updateUser}>Update User Info
       </button>
    </div>
    );}
```

03. Context API 사용예제

```
// UserInfo.js

import React, { useContext } from 'react';
import { UserContext } from './UserContext';

function UserInfo() {
    const { user } = useContext(UserContext);
    // UserContext에서 user 정보 가져오기

    return (
        <div>
            <p>Name: {user.name}</p>
            <p>Age: {user.age}</p>
        </div>
    );
}

export default UserInfo;
```